



Data Mining 1 Project Report

Exploring the BoardGames Dataset

A Study on Clustering, Classification,
Pattern Mining and Regression Techniques

Group Members

Fiza Naeem – mat. 704208
Lorena Sorce – mat. 598601
Nague Ivane Maeva – mat. 667328

Lecturers

Prof. Dino Pedreschi
Prof. Riccardo Guidotti
Dr. Andrea Fedele

Academic Year 2025–2026

Contents

1	Introduction	1
2	Data Understanding	1
2.1	Data Semantics	1
2.2	Distribution of the Variables and Statistics	1
2.2.1	Exploration of Numerical Variable Distributions	2
2.2.2	Exploration of Categorical Variable Distributions	2
3	Data Cleaning	3
3.1	Checking and Handling Duplicates in row and columns	3
3.2	Checking and Handling Missing Values	3
3.3	Checking and Handling 0	3
3.4	Outlier Detection and Handling	4
3.4.1	IQR for Outliers and Visualization	4
3.4.2	Log Transformation and Visualization	4
4	Clustering	4
4.1	Data preparation for Clustering or Feature Subset Selection	5
4.2	K-Means	5
4.2.1	Scaling	5
4.2.2	Feature Subset Selection and Determining the Optimal Number of Clusters (k)	5
4.2.3	Final Model	7
4.2.4	Visualization	7
4.2.5	Final Interpretation	8
4.3	Density-Based Clustering (DBScan)	8
4.3.1	Feature Selection	8
4.3.2	Preprocessing	8
4.3.3	Parameter Tuning: Choosing eps and minimum samples	9
4.3.4	Interpretation of Clusters	10
4.3.5	Conclusion	10
4.4	Hierarchical Clustering	10
4.5	Conclusion on best algorithm	11
5	Classification	11
5.1	Decision Tree Classification	11
5.1.1	Decision Tree Visualization	13
5.2	KNN	14
5.2.1	Preprocessing	14
5.2.2	Hyperparameter Optimization	14
5.2.3	Dimensionality Reduction	14
5.2.4	Per-Class Performance Analysis	14
5.2.5	Model Configuration and Class Imbalance Effects	15
5.2.6	Confusion Matrix (Tuned KNN)	15
5.2.7	PCA performance Trade-offs	15
5.2.8	Conclusion	16

5.3	Naive-Bayes Classification	16
6	Conclusion	16
7	Regression	17
8	Observations	17
8.1	Linear, Ridge, and Lasso Regression	18
8.2	Decision Tree Regression	18
8.3	K-Nearest Neighbors Regression	18
9	Pattern Mining	18
9.1	Data Preparation and Attribute Selection	18
9.2	Discretization and Transaction Construction	18
9.3	Apriori Support Threshold Analysis	19
9.4	Association Rule Extraction and Evaluation	19
9.5	Visualization	19
9.6	Summary	20
10	Limitations and Future Work	20
11	Conclusion	20

1 Introduction

This report examines a large board game dataset from an online community, comprising over 20,000 titles described by complexity, playtime, popularity, and structural features. It outlines variable definitions and statistics, the data-cleaning process, and clustering techniques used to uncover patterns. The analysis then applies supervised learning and regression models to evaluate predictive performance, followed by an exploration of frequent patterns and associations. The report concludes with a discussion of limitations and a summary of key findings.

2 Data Understanding

2.1 Data Semantics

The dataset provides a comprehensive overview of board games, it containing **21,925 records** and **46 attributes**. Knowing the data types is necessary because it provides us the information that how the values in each column are stored and interpreted. Semantic meaning of the variables is reported in the following tab (Tab 1).

Binary	Categorical	Ranks (Int)	Object	Numerical	
				Integer	Float
Kickstarted	Rating	Rank:strategygames	Name	YearPublished	GameWeight
IsReimplem.on	Family	Rank:abstracts	Description	MinPlayers	ComWeight
Cat:Thematic		Rank:familygames	ImagePath	MaxPlayers	ComAgeRec
Cat:Strategy		Rank:thematic	GoodPlayers	BestPlayers	LanguageEase
Cat:War		Rank:cgs		NumOwned	
Cat:Family		Rank:wargames		NumWant	
Cat:CGS		Rank:partygames		NumWish	
Cat:Abstract		Rank:childrensgames		NumWeightVotes	
Cat:Party				MfgPlaytime	
Cat:Childrens				ComMinPlaytime	
				ComMaxPlaytime	
				MfgAgeRec	
				NumUserRatings	
				NumComments	
				NumAlternates	
				NumExpansions	
				NumImplementations	

Table 1: Dataset variables grouped by core data type.

2.2 Distribution of the Variables and Statistics

This section analyzes the dataset's variables by examining their distributions and key statistical properties. It reviews numerical and categorical features to identify patterns or irregularities relevant to subsequent preprocessing, noting that the dataset is largely numeric with some textual fields unsuitable for direct numerical analysis. The dataset consists predominantly of integer-valued attributes (78.3%), followed by object-type variables (13.0%) and floating-point features (8.7%).

2.2.1 Exploration of Numerical Variable Distributions

To understand the shape of the distributions of the numerical variables, a histogram was generated for each numerical attribute in the dataset, as shown in Figure 1. Several types of behavior can be distinguished:

- **Variables with a right-skewed distribution** typical of counting variables. The distribution is therefore strongly skewed, and the variance is dominated by a small number of records.
- **Variables with discrete and range-bounded distribution**, in this case the distribution is balanced and moderately symmetrical with concentrations around average values.
- **Nearly normal or moderately skewed variables** such as YearPublished or Playtime, which represent natural temporal or quantitative phenomena.
- **Multimodal or irregular distributions** observed for MinPlayers, MaxPlayers and similar variables, where distinct groups of games with different player configurations emerge.
- **Sparse or degenerate distributions** present in binary variables characterized by a high concentration of zeros.

The following image shows six synthetic histograms that reproduce the major distribution shapes observed in the dataset:

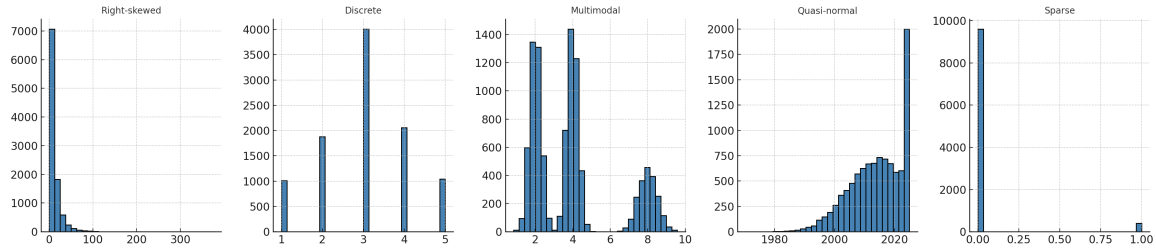


Figure 1: Synthetic histograms

2.2.2 Exploration of Categorical Variable Distributions

Text columns are not used directly in the models, while categorical attributes are reserved for qualitative analysis or encoding. The *GoodPlayers* variable is stored as an irregular text field with inconsistent formats and separators, often incomplete or empty, making it unsuitable for direct numeric use without extensive preprocessing and unreliable for modeling purposes.

3 Data Cleaning

3.1 Checking and Handling Duplicates in row and columns

The dataset was checked for duplicate rows. A duplicate check based on *Name* and *YearPublished* identified 12 rows corresponding to 6 games; however, since these entries differed in scores or thematic rankings, they were retained as valid variants. Overall, the analysis confirmed that there are **no exact row duplicates** in the dataset. During the inspection of duplicate columns, it was observed that *MfgPlayTime* and *ComMaxPlaytime*, despite representing different concepts, contain identical values for all observations. This was verified both through direct equality checks and via the correlation matrix, where the two variables exhibited a correlation of 1, indicating perfect overlap. Consequently, the *MfgPlayTime* column was removed, and *ComMaxPlaytime* was retained, as it better reflects community-based estimates and aligns more closely with the user-driven variables used in the analysis.

3.2 Checking and Handling Missing Values

The exploratory analysis of the dataset revealed a total of 26,701 missing values. The variable **“Family”** was the most affected, with 15,262 missing values, corresponding to 69.6% of the total.

The remaining variables showed a lower proportion of missing data. Specifically, **LanguageEase** presented 5,891 missing values (26.9%), while **ComAgeRec** had 5,530 (25.2%). The **ImagePath** variable showed only 17 missing values (0.1%), and **Description** had just one missing value (0.0%).

To improve reliability, variables with *more than 50% missing values were removed*; consequently, the *Family* variable was excluded despite its potential relevance. *LanguageEase* and *ComAgeRec*, which showed right-skewed distributions and outliers, were imputed using the median. Missing values in *ImagePath* and *Description* were replaced with the string “Unknown”. After these steps, the dataset contained no missing values.

3.3 Checking and Handling 0

In the dataset, some values equal to **0** were invalid for the variable type. We identified these cases by analyzing the distribution of zeros in numeric variables, *excluding binary variables* (Cat:, IsReimplementation, Kickstarted), for which zeros are legitimate. During this phase, we removed the **NumComments** column because it had a 100% zero percentage. The same operation was done for **BestPlayers** because it had 19,944 zeros, equal to 90.96%.

Zeros were treated as missing values and imputed according to the nature of each variable. **GameWeight**, **ComWeight**, and **NumWeightVotes** were replaced with their median values, while **MinPlayers** was set to 1 as a logical lower bound. **MaxPlayers** was imputed using the median within its category group, and **YearPublished** was replaced with the median. After the intervention, inconsistent zeros are no longer present, improving the data quality for subsequent analyses.

3.4 Outlier Detection and Handling

Boxplots were generated for numerical features (except for Binary variables and Rank:) in the training dataset to detect outliers using the interquartile range (IQR) method.

3.4.1 IQR for Outliers and Visualization

The identification was performed using the interquartile range (IQR) method, applied to all numerical variables in the dataset. For each column, the following were calculated: the first quartile (Q1, 25th percentile), the third quartile (Q3, 75th percentile), and the interquartile range ($IQR = Q3 - Q1$). A value was considered potentially anomalous if it fell outside the range:

$$[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$$

It compressed high skew feat

3.4.2 Log Transformation and Visualization

The variables *ComMinPlaytime*, *NumAlternates*, *MaxPlayers*, *NumExpansions*, and *NumWeightVotes* showed high levels of skewness, as shown by boxplots. To reduce the impact of extreme values and stabilize the distributions, a logarithmic transformation was applied. This improved distributional symmetry and scale stability, with before-and-after comparisons reported in *Figures 2 and 3*.

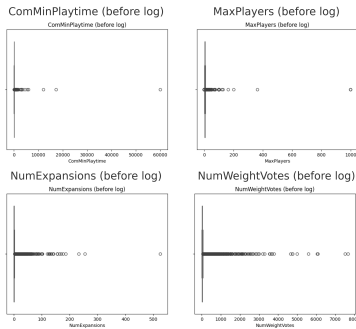


Figure 2: Boxplots before log transformation

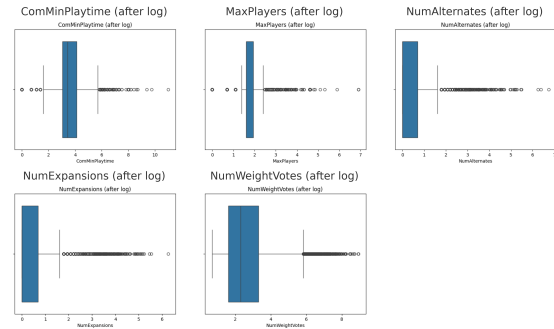


Figure 3: Boxplots after log transformation

moving to

4 Clustering

In this section, three complementary algorithms were selected: *k-means*, a centroid-based approach focused on compact and spherical clusters; *DBSCAN*, a density-based model suited for irregular shapes and noise detection; and *hierarchical* clustering, which captures nested structures and multilevel relationships in the data without requiring a predefined number of clusters.

4.1 Data preparation for Clustering or Feature Subset Selection

Before applying the clustering algorithms, the dataset underwent a set of standard preprocessing steps to ensure consistent scaling and reduce distortions driven by skewed distributions.

Since the dataset contains both numerical and categorical variables, we first explored the correlation matrix (Fig.4) to identify redundant or highly related attributes:

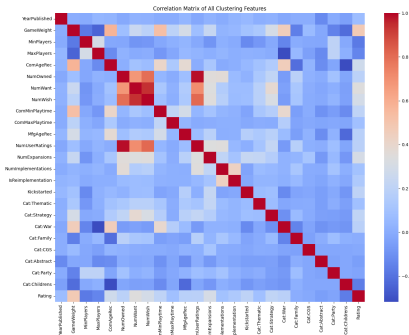


Figure 4: Correlation Matrix of All Clustering Features

Highly correlated feature pairs:

- NumOwned – NumUserRatings (0.985)
- NumWant – NumWish (0.940)
- NumWish – NumUserRatings (0.814)

To prevent overemphasizing popularity, **NumUserRatings** and **NumWish** were **removed**, retaining the more stable NumOwned and NumWant. Non-numeric and Euclidean-incompatible variables including textual identifiers, unstructured fields (Good-Players), non-informative variables, Rank, and binary features were excluded or adjusted. The Rating variable was converted to numeric form using Label Encoding (0, 1, 2).

4.2 K-Means

In this analysis, we compare the three scaled datasets with a little modification resulting from the data preparation phase. It is use to identify natural patterns in the dataset to obtain coherent sets to analyze and interpret.

4.2.1 Scaling

Since K-Means uses Euclidean distance, all continuous variables were scaled using **MinMaxScaler**. The result is a fully cluster-compatible numeric matrix.

4.2.2 Feature Subset Selection and Determining the Optimal Number of Clusters (k)

For the analysis we used both the **Elbow Method** and **Silhouette Score** for three cases, applying K-Means with k from 2 to 10. We constructed three different configurations of the dataset to determine which representation of the games produced the most stable and interpretable segments. Comparing the three subsets allowed us to assess

how cluster structures vary with different information and to select the most robust configuration for the final analysis.

- In the **first case** (Fig.5), using *all features* (23 variables), clustering quality stays low. The silhouette score peaks at 0.338 with $k=5$, indicating that mixing heterogeneous variables adds noise and weakens group separation.
- In the **second case** (Fig.6), when using only *cleaned and scalable numerical variables* (13 variables), the silhouette score increases to 0.378, but the model collapses the structure into two large, coarse clusters. The underlying complexity of the dataset is lost.
- The **third case** (Fig.7), based on a selected subset of core features (5 variables: *NumOwned*, *NumWant*, *ComMinPlaytime*, *ComMaxPlaytime*, *Rating*), delivers the most effective result. With $k=3$, the silhouette score reaches 0.579, significantly higher than the other scenarios. This configuration produces cleaner, more coherent, and genuinely interpretable clusters.

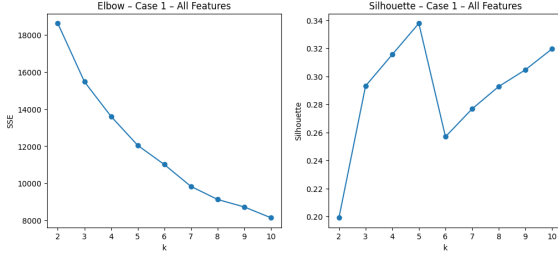


Figure 5: All variables

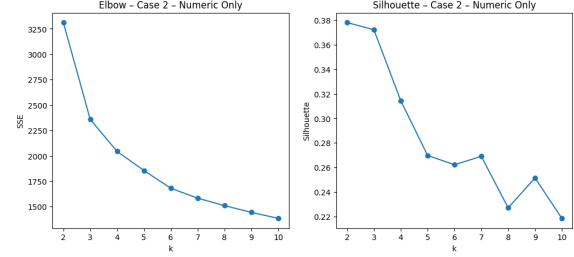


Figure 6: Only numerical variables

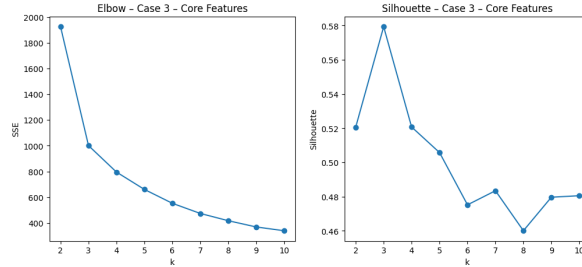


Figure 7: Subset of core features

- $k=2 \rightarrow \text{silhouette} = 0.5207$
- $k=3 \rightarrow \text{silhouette} = 0.5794 \leftarrow \text{the best}$
- $k=4 \rightarrow \text{silhouette} = 0.5208$
- $k=5 \rightarrow \text{silhouette} = 0.5058$

We selected **Case 3** and $k = 3$ as the final K-Means configuration as it is the best for obtaining balanced, stable and interpretable clusters.

4.2.3 Final Model

The final model was trained on: *NumOwned*, *NumWant*, *ComMinPlaytime*, *ComMaxPlaytime*, *Rating*. These features compactly represent the game’s popularity, user interest, and perceived complexity, making them particularly suited to building meaningful groups.

Each game was assigned to one of three clusters, leading to a balanced split:

- **Cluster 0: 7245 games**
Games owned by a small number of users, with near-zero ratings and short playtimes. These are marginal or niche titles.
- **Cluster 1: 9644 games**
Games with medium ownership and interest values, ratings around 0.5, and higher playtimes. Typically strategic and more challenging titles.
- **Cluster 2: 5036 games**
Very popular games, with a rating close to 1.0 and moderate complexity. They represent the most popular and well-known games.

Afterwards, the centroid analysis was done: **we can see a clear relationship by examining cluster centroid**

	NumOwned	NumWant	ComMinPlaytime	ComMaxPlaytime	Rating
0	0.003116	0.002621	0.302806	0.000801	-4.27e-14
1	0.007217	0.013876	0.325121	0.001328	0.500000
2	0.020078	0.059025	0.356812	0.002873	1.000000

Table 2: Cluster Centroids

clust 0 has lmh values around
has ratin close to ..corr to lmh rated gam

4.2.4 Visualization

To visually assess the quality of the clustering with **k=3**, PCA was applied in a three-dimensional space (*Fig 8*). The visualization shows three well-separated clusters occupying distinct regions of the PCA space, indicating a stable and coherent clustering structure.

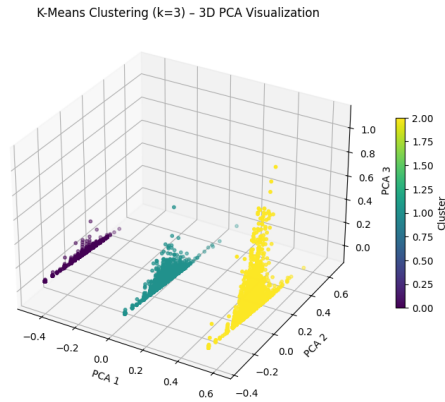


Figure 8: PCA 3D

4.2.5 Final Interpretation

In conclusion, the subset with the **5 core features** and $k = 3$ produced the best structure (silhouette 0.579), clearly distinguishing three groups: unpopular games, medium-complexity strategy games, and very popular and well-rated titles. These results indicate that user ownership, wishlists, and playtime metrics strongly define board game groups, while categorical labels were unnecessary for meaningful segmentation.

4.3 Density-Based Clustering (DBScan)

DBSCAN identifies clusters as dense areas in the feature space based on two parameters: **eps**, the maximum neighbor distance, and **minimum samples**, the minimum points needed to form a dense region. It classifies observations as *core points* (dense), *border points* (near dense areas), or *noise* (isolated).

4.3.1 Feature Selection

For the DBSCAN analysis, we selected **eight continuous gameplay-related variables**: *GameWeight*, *ComWeight*, *MinPlayers*, *MaxPlayers*, *ComAgeRec*, *LanguageEase*, *ComMinPlaytime*, and *ComMaxPlaytime*. These features capture fundamental aspects of game design, duration, accessibility, and group size-allowing density-based clustering to reflect real structural similarities.

4.3.2 Preprocessing

Before applying DBSCAN, we use **RobustScaler** for normalization, it reduces the influence of outliers for the heavy-tailed distributions. The resulting feature space behaved well, with a maximum absolute value on scale 11.01 and PCA scores ranging from approximately -7.9 to 11.8 .

To understand the relationships between features before clustering, we computed a correlation matrix.

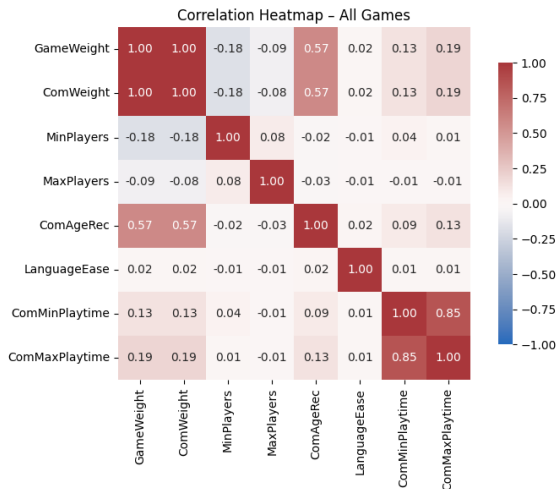


Figure 9: Correlation Heatmap – All Games

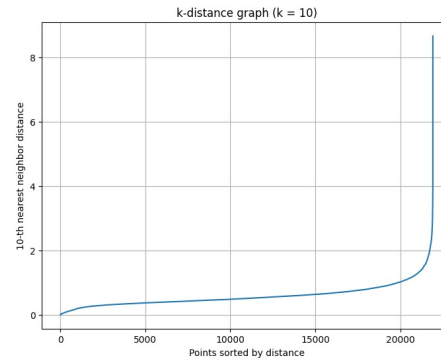


Figure 10: Elbow curve

The heatmap (Fig.9) shows strong correlations between complexity measures (*GameWeight*, *ComWeight*), strong correlation between minimum and maximum recommended play-time and weak correlations with player counts. This indicates that the selected features represent different dimensions of game design.

We started by reviewing the k-distance plot ($k = 10$) to frame a sensible tuning window. The curve (Fig.10) shows a clear elbow around 0.9, which is where larger neighborhood radii begin to merge distinct dense regions. That signal gave us a pragmatic boundary for the search, so we focused the eps sweep on the 0.75–0.90 range to keep the model calibration tight and efficient.

4.3.3 Parameter Tuning: Choosing eps and minimum samples

The tuning involved a grid of parameters:

- eps belong to 0.75, 0.80, 0.85, 0.90
- minimum samples belong to 3, 5, 8, 10

The **comparison** took into account the silhouette score, the number of clusters, the noise ratio, and the interpretation quality.

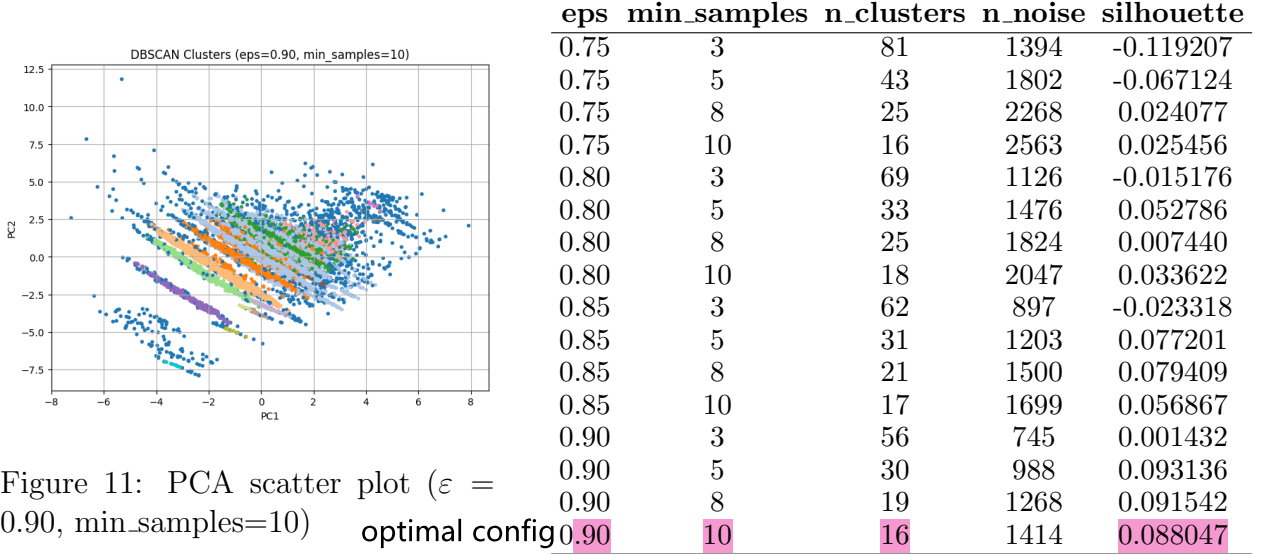


Figure 11: PCA scatter plot ($\varepsilon = 0.90$, min_samples=10)

optimal config 0.90 10 16 1414 0.088047

DBSCAN parameter results.

Finally, the best setup was: $\varepsilon = 0.90$ and *minimumsamples* = 10. This is the scenario that maximizes natural separation, produced the most moderate number of clusters, and maintains consistent density.

Noise was reasonable (1414 out of 21925, 6.45%) and Silhouette score was among the highest and most stable in the grid (0.088). With lower values of ε , DBSCAN produced too many micro-clusters, while higher values merged most observations into a single large cluster. The selected setting therefore represents the best trade-off between fragmentation and over-smoothing.

4.3.4 Interpretation of Clusters

DBSCAN identifies **bands** (Fig.11) of similar games driven by playtime, complexity, and recommended age, rather than compact clusters. The PCA visualization (Figure 12) reveals elongated and partially **overlapping** cluster shapes, This reveal substantial cluster overlap, indicating weak density-based separation. As a result, DBSCAN offers structural insight but *limited cluster clarity*.

4.3.5 Conclusion

DBSCAN separates the dataset into a large group of standard games, two intermediate clusters with recognizable profiles, and a set of outliers with extreme values, this highlights niche and anomalous cases. Despite this segmentation, DBSCAN fails to produce a balanced and clearly interpretable cluster structure.

4.4 Hierarchical Clustering

The dataset was analyzed using agglomerative hierarchical clustering. The selected numerical features were: *ComWeight*, *ComAgeRec*, *LanguageEase*, *ComMinPlaytime* and *ComMaxPlaytime*. This subset provides a balanced representation of gameplay characteristics while avoiding unnecessary redundancy and enabling effective distance-based clustering. All features were scaled using **RobustScaler** to ensure meaningful distance computations. Four linkage criteria (single, complete, average and Ward) were compared using **Euclidean** distance on a random sample of **400 observations**.

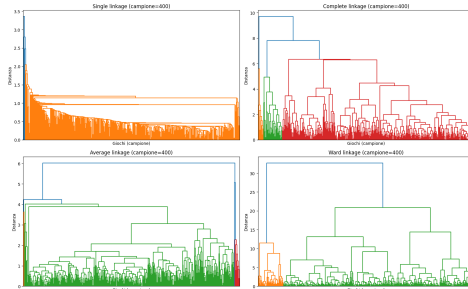


Figure 12: Dendrograms with four linkage methods

Table 3: Hierarchical clustering results with Euclidean distance.

Linkage	Clusters	Cut	Silhouette
Single	1	—	Not enough clusters
Average	2	6	0.603
Ward	3	15	0.241
Complete	2	8	0.603

C/A

The number of clusters (Tab. 3) was determined by selecting the cut threshold that maximized the silhouette score for each linkage method. **Single** and **average** linkage failed to identify meaningful structures (Fig.12), because single producing a flat dendrogram and collapsing all observations into a single cluster for all tested cut thresholds, and because average

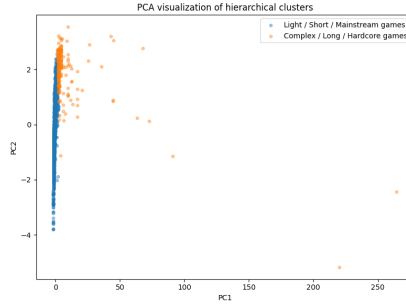


Figure 13: PCA visualization

linkage identifies some structure but rapidly merges all observation, providing limited interpretability.

Ward produced over-segmentation with moderate silhouette values. **Complete linkage** yielded a clear and well-separated structure, achieving the highest silhouette score, making it the optimal choice for this dataset.

The cut threshold was *not selected arbitrarily*. For each linkage method, a range of cut values was evaluated and the corresponding silhouette scores were computed, allowing the selection of the cut that maximizes cluster separation. Silhouette analysis indicated a strong optimum for two clusters (**silhouette 0.60**), corresponding to a cut threshold of approximately 8.

The **first cluster** appears to represent a small set of complex, long-duration games with higher age recommendations, while the **second** mainly includes lighter, mainstream titles, according to our interpretation.

A PCA-based visualization (Fig.13), used only for interpretative purposes, confirmed this separation, with the first two components explaining approximately 98% of the total variance.

4.5 Conclusion on best algorithm

K-Means emerged as the most effective clustering method, producing stable and well-balanced clusters with low computational cost and a silhouette score of 0.5794 for three clusters, making it the most suitable solution for this dataset.

5 for Classification

For the classification task, we employed Decision Tree, K-Nearest Neighbors (KNN), and Naive Bayes models using rating as the target variable.

5.1 Decision Tree Classification

This section analyzes the performance of a Decision Tree classifier for predicting the categorical target variable *Rating* (Low, Medium, High). Two configurations were evaluated: a baseline Decision Tree with default hyperparameters and a tuned Decision Tree optimized via cross-validation. The analysis was conducted on a cleaned dataset with no missing values, requiring no imputation. The target variable was numerically encoded solely for correlation analysis. Prior to modeling, the class distribution was examined and revealed moderate imbalance, with the Medium class being the most frequent. To mitigate this effect, all train-test splits and cross-validation procedures were stratified.

Baseline Model Performance The baseline Decision Tree, trained without explicit constraints achieved an *accuracy* of approximately **0.67** and a weighted *F1-score* close to **0.67**. Macro-averaged precision, recall, and F1-score were also balanced, indicating that performance was not driven solely by the majority class. The confusion matrix shows that most misclassifications occur between the Medium class and the two extreme classes, while Low and High ratings are rarely confused with each other. The one-vs-rest ROC curves confirm this behavior, with higher AUC values for Low and High ratings and lower discrimination for the Medium class.

Hyperparameter Tuning and Model Selection To improve performance, hyperparameter tuning was performed using **GridSearchCV** with stratified **3-fold cross-validation**. The search explored different splitting criteria (Gini, entropy), maximum tree depths (5, 10, 15, None), minimum samples per split (2, 5, 10), and minimum samples per leaf (1, 2, 4). (Fig. 14) summarizes the average cross-validation accuracy across *splitting criteria* and tree depths, while (Fig.15) provides a detailed view of *accuracy variations* for each parameter combination.

Tuned Model Performance The optimal configuration selected using the Gini criterion (max depth = 10, min samples split = 2, min samples leaf = 2) achieving a best cross-validation accuracy of approximately **0.703**. When evaluated on the held-out test set, the tuned model achieved an *accuracy* of **0.709**, with a macro-averaged *precision* of **0.726**, *recall* of **0.697**, and *F1-score* of **0.708**. Compared to the baseline, the tuned model shows improved class balance and more reliable discrimination. The confusion matrix indicates stronger predictions for Low and High ratings, while most remaining errors involve the Medium class. ROC analysis confirms high AUC values for Low (0.906) and High (0.886) ratings, with a lower but still strong AUC for Medium (0.795).

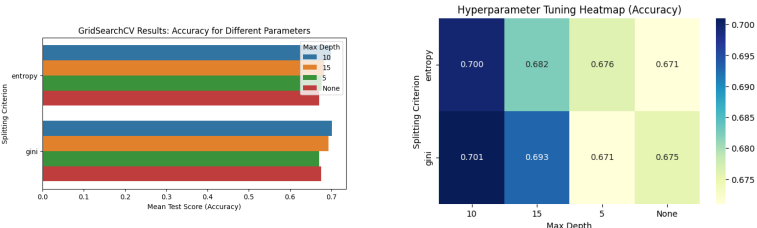


Figure 14: GridSearch accuracy by criterion and depth.

Figure 15: Hyperparameter tuning heatmap (accuracy).

Figure 16: Decision Tree baseline vs tuned performance and hyperparameter tuning results.

Comparison and Interpretation Overall, **hyperparameter tuning** leads to a clear *improvement over the baseline model*, particularly in terms of balanced class performance and interpretability. The tuned model achieves stronger discrimination for extreme ratings while handling intermediate cases as effectively as expected given feature overlap.

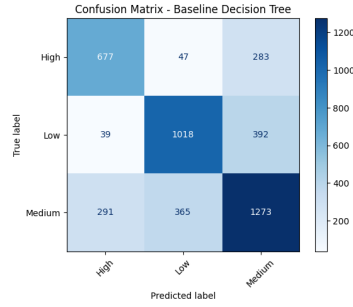


Figure 17: Confusion matrix (baseline Decision Tree).

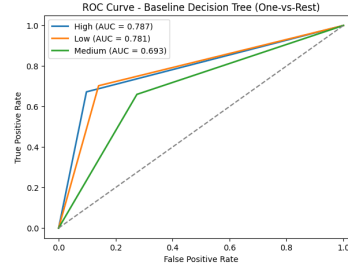


Figure 18: One-vs-rest ROC curves (baseline Decision Tree).

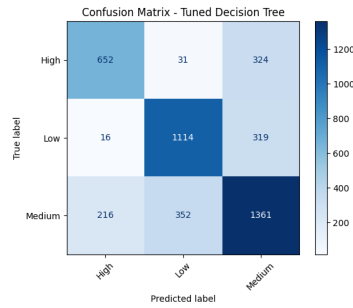


Figure 19: Confusion matrix (tuned Decision Tree).

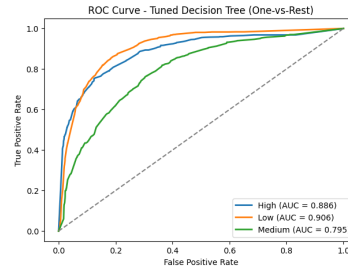


Figure 20: One-vs-rest ROC curves (tuned Decision Tree).

5.1.1 Decision Tree Visualization

The tuned Decision Tree was visualized by restricting the displayed depth to four levels. The upper splits show that popularity-related features such as NumWish, NumWant, and Rank: wargames dominate early decision-making, indicating their strong influence on the predicted rating. Deeper splits incorporate additional attributes such as publication year and complexity, refining class separation.

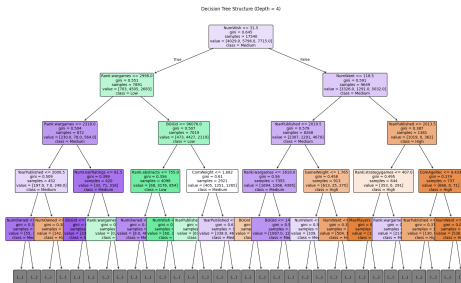


Figure 21: Visualization of the tuned Decision Tree classifier truncated at depth 4 for interpretability

5.2 KNN

The dataset exhibits class imbalance across the three rating categories: Medium: 9,644 games (43.99%), Low: 7,245 games (33.04%), High: 5,036 games (22.97%). This imbalance suggests that *most games fall into the medium rating category*, with high-rated games being relatively rare. This distribution pattern influenced model performance across classes.

5.2.1 Preprocessing

A streamlined preprocessing pipeline was applied for classification, including correlation analysis to remove highly correlated features ($r > 0.8$), feature selection to drop non-informative columns (identifiers, text fields, image paths), and label encoding of the target variable. All features were standardized with StandardScaler, and the data were split into training and test sets (80/20) using a fixed random seed (42) for reproducibility.

5.2.2 Hyperparameter Optimization

GridSearchCV with 5-fold cross-validation tested 24 configurations across three parameters:

- **Number of Neighbors (K):** Tested values [7, 11, 15, 21, 25, 31]
- **Distance Metric:** Evaluated euclidean and manhattan
- **Weight Function:** Compared uniform and distance-based weighting

5.2.3 Dimensionality Reduction

Principal Component Analysis (PCA) was applied to reduce feature dimensionality while preserving 95% of variance, transforming 18 features into 16 principal components. This technique aims to improve computational efficiency and reduce overfitting risk.

Table 4: KNN Performance Summary

Model	Configuration	Test Accuracy
Baseline KNN	K = 21, default	61.82%
Tuned KNN	K = 21, Manhattan distance	62.39%
KNN with PCA	16 components, 95% variance	61.46%

Key Findings:

- Hyperparameter tuning improved *accuracy* by **0.57** percentage points
- Optimal configuration: K=21, Manhattan distance, distance-based weighting
- PCA maintains performance within 1% while reducing features by 11%

5.2.4 Per-Class Performance Analysis

Classification performance varies significantly across rating categories: Low and Medium

Table 5: Classification Report for Tuned KNN

Class	Precision	Recall	F1-Score
High	0.677	0.512	0.583
Low	0.631	0.654	0.642
Medium	0.588	0.647	0.616

categories achieve similar recall rates (65.4% and 64.7% respectively), while the High rating class shows lower recall (51.2%) but higher precision (67.7%). This suggests the model is more conservative when predicting high ratings, reducing false positives at the cost of missing some true high-rated games.

5.2.5 Model Configuration and Class Imbalance Effects

GridSearchCV identified an optimal KNN configuration with $K = 21$, Manhattan distance, and distance-based weighting, which outperformed Euclidean distance and uniform weighting. The Manhattan metric proved more robust to outliers, while distance-based weighting improved classification by assigning greater influence to closer neighbors. However, class imbalance affects performance, particularly for the High rating class, which represents only 23% of the dataset. This class exhibits lower recall (51.2%), indicating missed true positives, while maintaining higher precision (67.7%), reflecting more conservative but accurate predictions.

5.2.6 Confusion Matrix (Tuned KNN)

The confusion matrix for the tuned KNN model ($K=21$, Manhattan distance, distance weighting) shows the classification results across all three rating classes: The diagonal elements rep-

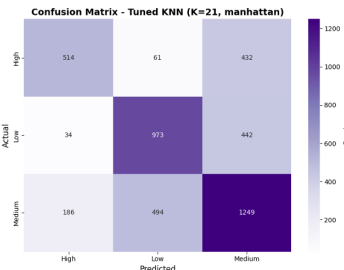


Figure 22: Confusion Matrix for KKN classification

resent correct classifications: 514 High, 973 Low, and 1249 Medium games correctly predicted. Notable misclassification patterns include 324 High games predicted as Medium and 636 Low games predicted as Medium, reflecting the model’s tendency to favor the *majority Medium class*.

5.2.7 PCA performance Trade-offs

The PCA analysis demonstrates that dimensionality reduction from 18 to 16 features results in only a 0.94% accuracy decrease. However, test accuracy decreased to 61.46% (-0.94%), indicating that the removed variance contained discriminative information for classification. The PCA analysis reveals:

Metric	Original Features	With PCA
Number of Features	18	16
Variance Retained	100%	95%
Test Accuracy	62.39%	61.46%
Dimensionality Reduction	—	11.1%

5.2.8 Conclusion

he tuned KNN model achieved 62.39% accuracy, demonstrating moderate predictive capability for board game rating classification. Manhattan distance with distance-based weighting proved optimal, while PCA showed marginal performance trade-offs.

5.3 Naive-Bayes Classification

This section analyzes the performance of Naive-Bayes classifier for predicting the categorical target variable Rating (Low, Medium, High). For the classification task, Gaussian Naive Bayes was used after selecting numerical features, binarizing Rank: variables, removing non-informative (GoodPlayers) and highly correlated attributes (ComWeight, NumWish, NumUserRatings), and using Label Encoding on target (Rating). The dataset was split into training 60%, validation 20%, and test 20% sets. The **baseline** GaussianNB model achieved approximately 53% accuracy, but struggled to correctly classify the Medium class (most games fall into the medium rating category), which was frequently confused with Low, indicating a strong class bias. To improve the model's performance, three alternative approaches were evaluated:

Random Forest for feature selection Slightly improves overall accuracy and precision for the *High* class, but does not significantly reduce class confusion, particularly between *Medium* and *Low*.

SMOTE (train only) It was applied to the training set to balance class distribution. This strategy reduced the bias toward the majority class (Medium) and improved the model's performance on underrepresented classes.

PCA (95% variance explained)

It was used to reduce feature correlations while preserving most of the data variance. Although overall accuracy remained comparable to the baseline, this configuration produced **more balanced predictions** across classes, resulting in a more uniform confusion matrix. Although NB + SMOTE achieved slightly higher accuracy, the NB + PCA configuration exhibited a more balanced confusion matrix (Fig.24), with a more uniform distribution of correct predictions across classes. This suggests that PCA improved model stability and reduced class bias, at the cost of marginally lower discriminative power. Although the absolute accuracy (0.5318) is modest, ROC analysis (Fig. 23) reveals relatively strong AUC values, suggesting that the model provides informative probability estimates and retains meaningful ranking capability, despite accuracy being assessed at a single operating threshold. This trade-off indicates that PCA prioritizes robustness and class balance over marginal gains in discriminative power (Fig.23). As seen in (Fig.24), the confusion matrix visualizes the number of true positives, false positives, true negatives, and false negatives, providing insights into the classification errors made by the model.

6 Conclusion

The **Decision Tree** achieved the best balance particularly after hyperparameter tuning, with strong discrimination for extreme rating classes. KNN provided competitive accuracy but remained sensitive to class imbalance, favoring the majority Medium class despite tuning and PCA-based dimensionality reduction. Naive Bayes showed the weakest performance overall,

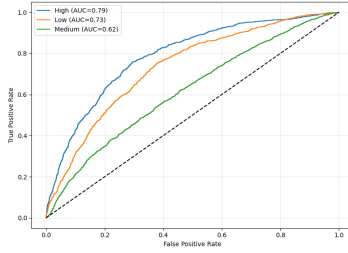


Figure 23: ROC curve for NB + PCA

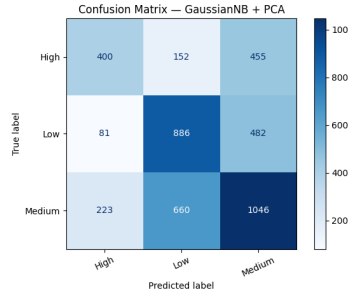


Figure 24: Confusion matrix for NB + PCA

tableClassification report for NB + PCA (Test set)			
Class	Precision	Recall	F1-score
High	0.57	0.40	0.47
Low	0.52	0.61	0.56
Medium	0.53	0.54	0.53

with strong class bias and limited discriminative power, though PCA and SMOTE improved prediction balance at the cost of accuracy.

7 Regression

A regression analysis was conducted to examine the relationship between the target variable Rating and selected numeric predictors. Although Rating is inherently categorical, it was treated as an ordinal numeric variable to enable the use of regression techniques and to assess whether linear and non-linear models could capture meaningful trends under both univariate and multivariate settings. Three feature configurations were evaluated:

1. **Univariate**: Uses NumWish as the single predictor.
2. **Multivariate (2)**: Uses NumWish and NumUserRatings.
3. **Multivariate (3)**: Uses NumWish, NumUserRatings, and NumOwned.

In addition, a configuration using all available numeric variables was included to assess whether a richer feature space improves predictive performance. The regression models considered include Linear Regression, Ridge Regression, Lasso Regression, Decision Tree Regression, and K-Nearest Neighbors (KNN). Model performance was evaluated on a held-out test set using R^2 , Mean Absolute Error (MAE), and Mean Squared Error (MSE).

Table 7: Regression results (R^2 , MAE, MSE) across models and feature configurations.

Model	All Variables	Univariate	Multivariate (2)	Multivariate (3)
Linear	0.557, 0.397, 0.246	0.091, 0.579, 0.505	0.101, 0.575, 0.499	0.102, 0.576, 0.499
Ridge	0.541, 0.404, 0.255	0.091, 0.579, 0.505	0.101, 0.575, 0.499	0.102, 0.576, 0.499
Lasso	0.421, 0.454, 0.321	0.091, 0.579, 0.505	0.101, 0.575, 0.500	0.103, 0.576, 0.498
Decision Tree	0.351, 0.330, 0.360	0.281, 0.506, 0.399	0.016, 0.511, 0.547	-0.104, 0.508, 0.613
KNN	0.435, 0.427, 0.314	0.182, 0.544, 0.454	0.323, 0.474, 0.376	0.334, 0.465, 0.370

8 Observations

Table 7 summarizes the regression results across all models and feature configurations. Overall, consistently low R^2 values indicate that the selected predictors explain only a limited portion of the variance in the target variable.

8.1 Linear, Ridge, and Lasso Regression

Linear, Ridge, and Lasso regressions show very similar performance across univariate and multivariate settings. Low R^2 values and stable MAE/MSE indicate a weak linear relationship between the predictors and the target variable.

8.2 Decision Tree Regression

Decision Tree regression achieves slightly higher R^2 when using all available variables, suggesting limited non-linear effects. However, the overall improvement is marginal and not sufficient for reliable regression performance.

8.3 K-Nearest Neighbors Regression

KNN regression performs poorly in most configurations, with negative R^2 values in several cases. This indicates that local similarity among instances does not provide meaningful predictive power for this task.

9 Pattern Mining

Pattern mining was conducted to complement the predictive models by providing a descriptive analysis of the relationships among game characteristics. By applying association rule mining techniques, this analysis seeks to identify interpretable patterns.

9.1 Data Preparation and Attribute Selection

For the pattern mining task, a subset of the original dataset was selected in order to retain variables that are meaningful for association analysis. In total, nine attributes were used:

- **Numerical attributes:** `GameWeight`, `NumUserRatings`, `NumOwned`, `NumWish`
- **Binary attributes:** `Kickstarted`, `IsReimplementation`
- **Categorical attributes:** `Cat:Strategy`, `Cat:Thematic`
- **Target variable:** `Rating`

This selection balances game complexity indicators, popularity-related variables, thematic information, and user evaluation outcomes, allowing the discovery of interpretable relationships between game characteristics and ratings.

9.2 Discretization and Transaction Construction

Since association rule mining requires categorical data, numerical attributes were discretized using *quantile-based binning*. This approach ensures that each bin contains approximately the same number of observations, reducing sparsity and producing more balanced transactions compared to equal-width binning.

Discretization was applied only to the numerical attributes *GameWeight*, *NumUserRatings*, *NumOwned*, and *NumWish*. Multiple binning configurations were explored, specifically 2-bin, 3-bin, and 4-bin discretizations, to assess the impact on rule stability and interpretability.

The 2-bin configuration was found to be overly coarse, merging distinct game profiles into broad categories. Conversely, the 4-bin configuration produced a large number of low-support items, leading to fragmented rules and reduced interpretability. The 3-bin configuration provided the best trade-off between information retention and rule clarity and was therefore selected. After discretization, all attributes were transformed into transactional format using one-hot encoding, where each transaction represents the presence or absence of specific attribute-value pairs (e.g., `NumOwned_bin_2`, `GameWeight_bin_1`). This process resulted in a high-dimensional transactional dataset containing over **11,000 binary items**, enabling the application of the Apriori algorithm.

9.3 Apriori Support Threshold Analysis

The Apriori algorithm was applied using multiple minimum support thresholds (10%, 5%, 2%, and 1%) to analyze how the number and complexity of frequent itemsets evolve.

At higher support thresholds (10%), the algorithm produced a limited number of highly general itemsets, capturing only the most common attribute combinations. As the support threshold decreased, the number of frequent itemsets increased substantially. Specifically, approximately **196 frequent itemsets** were identified at 10% support, compared to **409** at 5%, **912** at 2%, and **1,506** at 1% support.

This rapid growth illustrates the trade-off between coverage and interpretability: while lower support thresholds uncover richer patterns, they also increase rule complexity. Based on this analysis, a minimum support threshold of **2%** was selected for detailed rule exploration, as it yielded a sufficiently rich yet manageable set of patterns.

9.4 Association Rule Extraction and Evaluation

From the frequent itemsets obtained at the 2% support level, association rules were generated and evaluated using multiple quality measures, including *support*, *confidence*, *lift*. Particular attention was given to rules involving the *Rating* variable, as these provide insight into how combinations of game attributes relate to user evaluations.

Rules were analyzed under different ranking criteria. High-lift rules revealed strong non-trivial associations, with several rules exhibiting lift values above **7**, indicating dependencies well beyond chance co-occurrence. High-confidence rules achieved confidence values exceeding **0.99**, demonstrating strong predictive reliability, although not necessarily high lift. Additionally, rules ranked by leverage highlighted combinations with substantial real-world impact, particularly those involving popularity indicators such as *NumOwned* and *NumUserRatings*.

Across these rankings, combinations of ownership and user rating bins consistently appeared in high-quality rules, often in conjunction with thematic or strategic categories, suggesting a strong relationship between game popularity, thematic design, and user ratings.

9.5 Visualization

To complement the quantitative analysis, scatter plots were used to visualize relationships between key rule metrics. The *Lift vs. Support* plot shows that the highest lift values occur at low support levels, indicating strong but infrequent associations, while higher-support rules tend to exhibit more moderate lift.

Similarly, the *Confidence vs. Lift* plot shows that high confidence does not necessarily imply high lift. Several rules achieve near-perfect confidence with lift values close to 2, emphasizing the need to evaluate multiple metrics jointly when assessing rule quality.

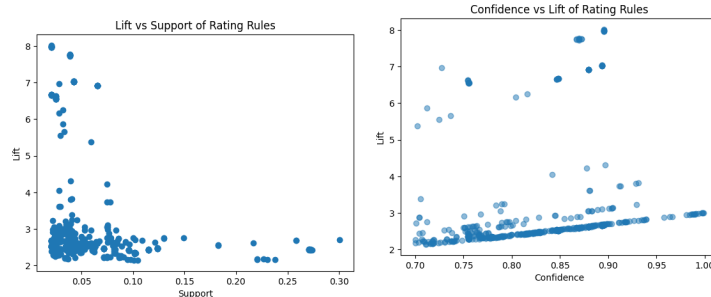


Figure 25: Lift versus support for association rules

Figure 26: Confidence versus lift for association rules

9.6 Summary

Overall, the pattern mining analysis reveals meaningful associations between game popularity, complexity, thematic categories, and user ratings. While highly frequent patterns tend to be broad and descriptive, lower-support rules uncover stronger and more specific relationships. The combined use of multiple evaluation metrics and visual analysis ensures that the extracted rules are both statistically sound and interpretable. These descriptive insights complement the predictive models developed in earlier sections by providing an interpretable view of the structural relationships present in the data.

10 Limitations and Future Work

This study provides a comprehensive analysis of a large board game dataset, but several limitations should be acknowledged. A key limitation concerns the imbalance in the target variable (Rating), with Medium-rated games being the most frequent class. Despite the use of stratified sampling and balanced evaluation metrics, all classification models showed difficulty in accurately separating the Medium class, indicating intrinsic overlap in feature distributions. Another limitation is the strong influence of popularity-related variables (e.g., NumOwned, NumWant, NumWish), which dominate both clustering and classification results and may obscure finer gameplay characteristics. Future work could explore domain-specific feature engineering and ordinal or ensemble classification models to better capture subtle rating differences. Additionally, regression models exhibited limited explanatory power, suggesting that treating Rating as a continuous variable is not ideal. Future studies may benefit from alternative formulations or more expressive modeling approaches. Finally, the pattern mining analysis relied on discretization, which may introduce information loss; adaptive or temporal pattern mining techniques could provide richer insights.

11 Conclusion

This project analyzed a large board game dataset through data cleaning, clustering, classification, regression, and pattern mining. After addressing missing values, outliers, skewed distributions, and redundant features, the dataset was prepared for reliable analysis. Among clustering methods, **K-Means achieved the best performance**, producing three balanced and interpretable clusters with a **silhouette score of 0.579**. In the classification task, the tuned **Decision Tree** emerged as the most effective model, achieving the best overall balance between accuracy, class-wise performance, and interpretability. KNN and Naive Bayes served as useful

benchmarks but showed lower or less stable performance. Regression analysis produced weak results, highlighting the difficulty of predicting subjective ratings using continuous models. In contrast, association rule mining revealed meaningful relationships between game popularity, complexity, thematic categories, and user ratings, complementing the predictive models with interpretable insights. Overall, this work provides a solid analytical framework for studying complex, user-driven datasets and highlights opportunities for future improvements in feature design and modeling strategies.